

Master Memory Manager API Documentation

Overview

The Master Memory Manager (MMM) provides a comprehensive API for memory management, system optimization, and advanced AI capabilities within the BDI framework.

Core API Categories

1. Master Control API

Central orchestration and system-wide control functions.

```
// Master Control System
typedef struct {
    uint64_t system_id;
    uint32_t component_count;
    uint32_t active_threads;
    uint64_t total_memory;
    uint32_t optimization_level;
} mmm_master_control_t;

// Initialize master control system
int mmm_master_control_init(mmm_master_control_t *control);

// Start system orchestration
int mmm_orchestrator_start(mmm_master_control_t *control);

// Shutdown system gracefully
int mmm_orchestrator_shutdown(mmm_master_control_t *control);

// Get system status
int mmm_get_system_status(mmm_master_control_t *control, mmm_system_status_t *status);
```

2. Memory Management API

Advanced memory allocation, deallocation, and optimization functions.

```
// Memory pool management
typedef struct {
    void *pool_base;
    size_t pool_size;
    size_t allocated;
    size_t free_blocks;
    uint32_t optimization_flags;
} mmm_memory_pool_t;

// Create optimized memory pool
mmm_memory_pool_t* mmm_create_memory_pool(size_t size, uint32_t flags);

// Allocate memory with optimization hints
void* mmm_alloc_optimized(mmm_memory_pool_t *pool, size_t size, uint32_t hints);

// Free memory with analytics
int mmm_free_optimized(mmm_memory_pool_t *pool, void *ptr);

// Memory pool statistics
int mmm_get_pool_stats(mmm_memory_pool_t *pool, mmm_pool_stats_t *stats);
```

3. Performance Optimization API

Real-time performance monitoring and optimization functions.

```
// Performance metrics
typedef struct {
    uint64_t cache_hits;
    uint64_t cache_misses;
    uint64_t memory_bandwidth;
    uint32_t cpu_utilization;
    uint32_t memory_utilization;
    double optimization_score;
} mmm_performance_metrics_t;

// Start performance monitoring
int mmm_performance_monitor_start(void);

// Get current performance metrics
int mmm_get_performance_metrics(mmm_performance_metrics_t *metrics);

// Trigger adaptive optimization
int mmm_trigger_optimization(uint32_t optimization_type);

// Set performance targets
int mmm_set_performance_targets(mmm_performance_targets_t *targets);
```

4. AI Capabilities API

Advanced AI-driven system management and optimization.

```
// AI system configuration
typedef struct {
    uint32_t model_type;
    uint32_t learning_rate;
    uint32_t prediction_window;
    uint32_t anomaly_threshold;
} mmm_ai_config_t;

// Initialize AI capabilities
int mmm_ai_init(mmm_ai_config_t *config);

// Predictive analytics
int mmm_predict_system_behavior(mmm_prediction_t *prediction);

// Anomaly detection
int mmm_detect_anomalies(mmm_anomaly_report_t *report);

// Self-healing trigger
int mmm_trigger_self_healing(uint32_t issue_type);

// Continuous learning update
int mmm_update_learning_model(mmm_learning_data_t *data);
```

5. Security & Compliance API

Enterprise-grade security and audit functions.

```
// Security configuration
typedef struct {
    uint32_t encryption_level;
    uint32_t access_control_flags;
    uint32_t audit_level;
    char security_policy[256];
} mmm_security_config_t;

// Initialize security framework
int mmm_security_init(mmm_security_config_t *config);

// Access control validation
int mmm_validate_access(uint32_t user_id, uint32_t resource_id, uint32_t operation);

// Audit logging
int mmm_audit_log(uint32_t event_type, const char *description);

// Compliance reporting
int mmm_generate_compliance_report(mmm_compliance_report_t *report);
```

6. Monitoring & Telemetry API

Real-time system monitoring and telemetry collection.

```
// Telemetry data structure
typedef struct {
    uint64_t timestamp;
    uint32_t metric_type;
    double value;
    char description[128];
} mmm_telemetry_data_t;

// Start telemetry collection
int mmm_telemetry_start(uint32_t collection_interval);

// Collect telemetry data
int mmm_collect_telemetry(mmm_telemetry_data_t *data, size_t max_entries);

// Real-time monitoring
int mmm_monitor_realtime(mmm_monitor_callback_t callback);

// Alert system
int mmm_set_alert_threshold(uint32_t metric_type, double threshold);
```

7. Configuration Management API

Dynamic configuration and feature flag management.

```
// Configuration structure
typedef struct {
    char key[64];
    char value[256];
    uint32_t flags;
    uint64_t timestamp;
} mmm_config_entry_t;

// Load configuration
int mmm_config_load(const char *config_file);

// Get configuration value
int mmm_config_get(const char *key, char *value, size_t value_size);

// Set configuration value
int mmm_config_set(const char *key, const char *value, uint32_t flags);

// Feature flag management
int mmm_feature_flag_set(const char *feature, bool enabled);
bool mmm_feature_flag_get(const char *feature);
```

Error Codes

```
#define MMM_SUCCESS 0
#define MMM_ERROR_INVALID_PARAM -1
#define MMM_ERROR_OUT_OF_MEMORY -2
#define MMM_ERROR_SYSTEM_FAILURE -3
#define MMM_ERROR_ACCESS_DENIED -4
#define MMM_ERROR_NOT_INITIALIZED -5
#define MMM_ERROR_TIMEOUT -6
#define MMM_ERROR_AI_FAILURE -7
#define MMM_ERROR_SECURITY_VIOLATION -8
```

Usage Examples

Basic System Initialization

```
// Initialize master control
mmm_master_control_t control;
if (mmm_master_control_init(&control) != MMM_SUCCESS) {
    // Handle initialization error
}

// Start orchestration
if (mmm_orchestrator_start(&control) != MMM_SUCCESS) {
    // Handle startup error
}
```

Memory Pool Management

```
// Create optimized memory pool
mmm_memory_pool_t *pool = mmm_create_memory_pool(1024*1024, MMM_POOL_OPTIMIZED);

// Allocate memory with cache hints
void *ptr = mmm_alloc_optimized(pool, 4096, MMM_HINT_CACHE_FRIENDLY);

// Use memory...

// Free with analytics
mmm_free_optimized(pool, ptr);
```

AI-Driven Optimization

```
// Initialize AI capabilities
mmm_ai_config_t ai_config = {
    .model_type = MMM_AI_PREDICTIVE,
    .learning_rate = 100,
    .prediction_window = 1000,
    .anomaly_threshold = 95
};
mmm_ai_init(&ai_config);

// Get predictions
mmm_prediction_t prediction;
mmm_predict_system_behavior(&prediction);

// Trigger optimization based on predictions
if (prediction.optimization_needed) {
    mmm_trigger_optimization(MMM_OPT_MEMORY_LAYOUT);
}
```

Integration Guidelines

BDI Kernel Integration

- Use `mmm_bdi_kernel_register()` to register with BDI kernel
- Implement callback functions for kernel events
- Follow BDI memory management protocols

HAL Framework Integration

- Initialize HAL components before MMM initialization
- Use HAL abstraction for hardware-specific operations
- Maintain compatibility with HAL versioning

Performance Considerations

- Use memory pools for frequent allocations
- Enable AI optimization for production workloads
- Monitor telemetry data for performance tuning
- Set appropriate cache hints for memory operations

Thread Safety

All MMM APIs are thread-safe unless explicitly noted. Use appropriate locking mechanisms when accessing shared resources from multiple threads.

Version Compatibility

This API documentation is for MMM Phase 4 (LEGENDARY BDI BUILD). Backward compatibility is maintained with Phase 1-3 APIs.